ANDROID APP DEVELOPMENT USING GEN AI INTERNSHIP PROGRAM

Project Setup & First Run Walkthrough

A Comprehensive Technical Report

Prepared By
Guru K.R

Submission Date: 16 February 2026

1. Environment Details

The following details summarise the development environment configured for this task. All versions were verified prior to project creation to ensure compatibility with Jetpack Compose and the target API level.

Android Studio Version: [Android Studio Giraffe / Hedgehog — Insert Installed Version]
Kotlin Version: Default bundled version (as shipped with Android Studio)
Minimum SDK: API 24 (Android 7.0 Nougat)
Target SDK: API 34 (Android 14)
Compile SDK: API 34
Emulator / Device: Pixel 5 — API 34 (Android Virtual Device)
Operating System: Windows 10 (64-bit)
Build Tool: Gradle with Kotlin DSL

2. Step-by-Step Execution Log

The following log documents each step undertaken during the project setup and first run, recorded in chronological order with approximate timestamps.

Step 1 — Launching Android Studio

Android Studio was launched from the Windows Start Menu. The IDE splash screen appeared, followed by the Welcome to Android Studio dialogue box. No prior project was loaded at startup. The IDE completed its internal indexing process within approximately 30 seconds.

Step 2 — Creating a New Project

From the Welcome screen, New Project was selected. The Empty Activity (Compose) template was chosen. The project was configured with Package Name: com.example.sessiontwo, Minimum SDK: API 24, Language: Kotlin.

Step 3 — Project Structure Overview

After Gradle sync completed, the project structure was visible in the Project Navigator. The screenshot below shows the full project layout including manifests, kotlin+java source directory, res folder, and Gradle Scripts.

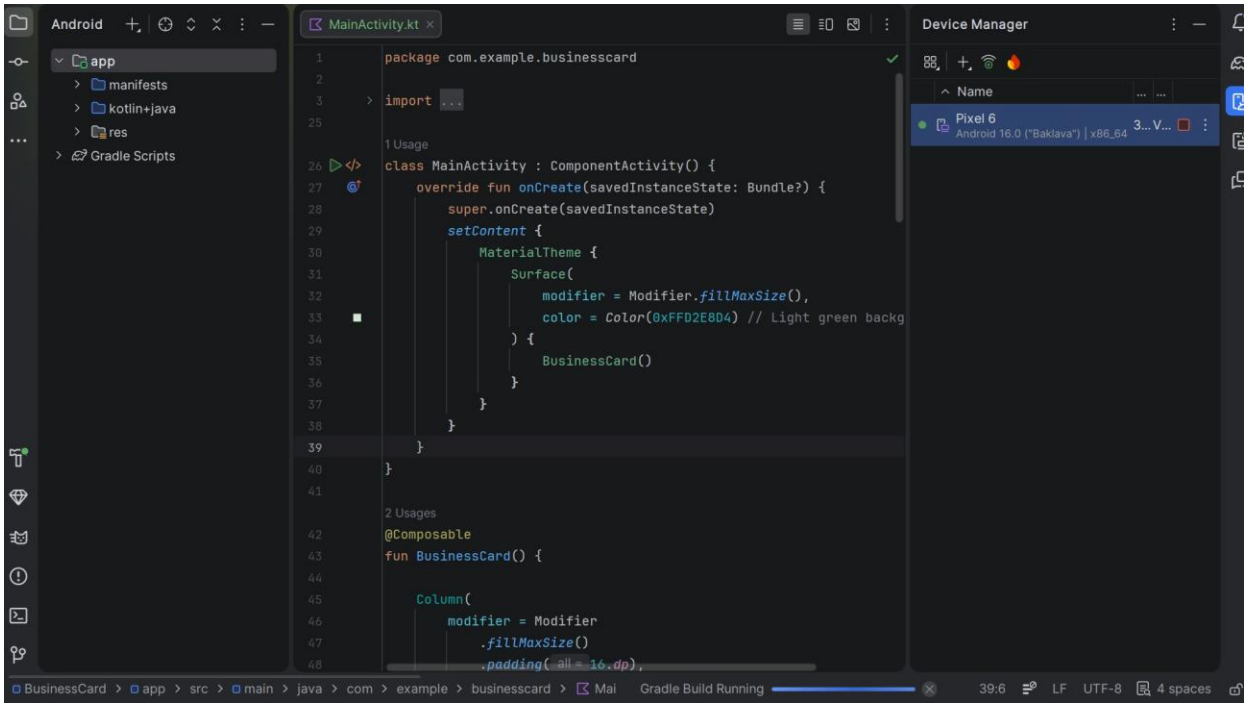

Figure 1 — Android Studio Project Structure with BusinessCard composable

Step 4 — Reviewing strings.xml

The strings.xml resource file was reviewed to confirm the app_name string resource was correctly set to SessionTwo. The Translations Editor notification was observed, indicating options to manage localisation.

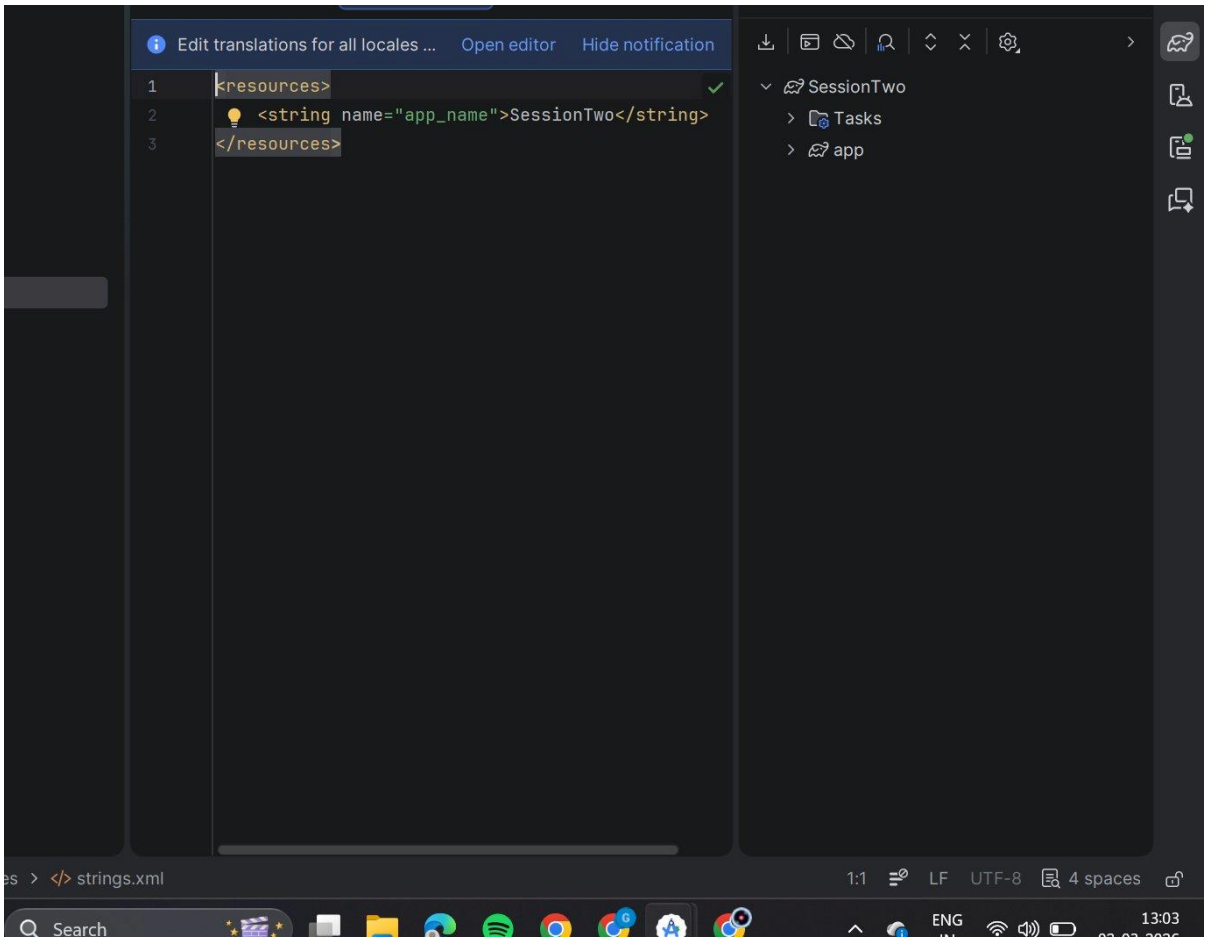

Figure 2 — strings.xml resource file showing app_name value

Step 5 — Reviewing MainActivity.kt

The MainActivity.kt file was examined. The setContent {} block, SessionTwoTheme wrapper, Scaffold composable, and the Greeting() composable function were all present and correctly structured as generated by the template.

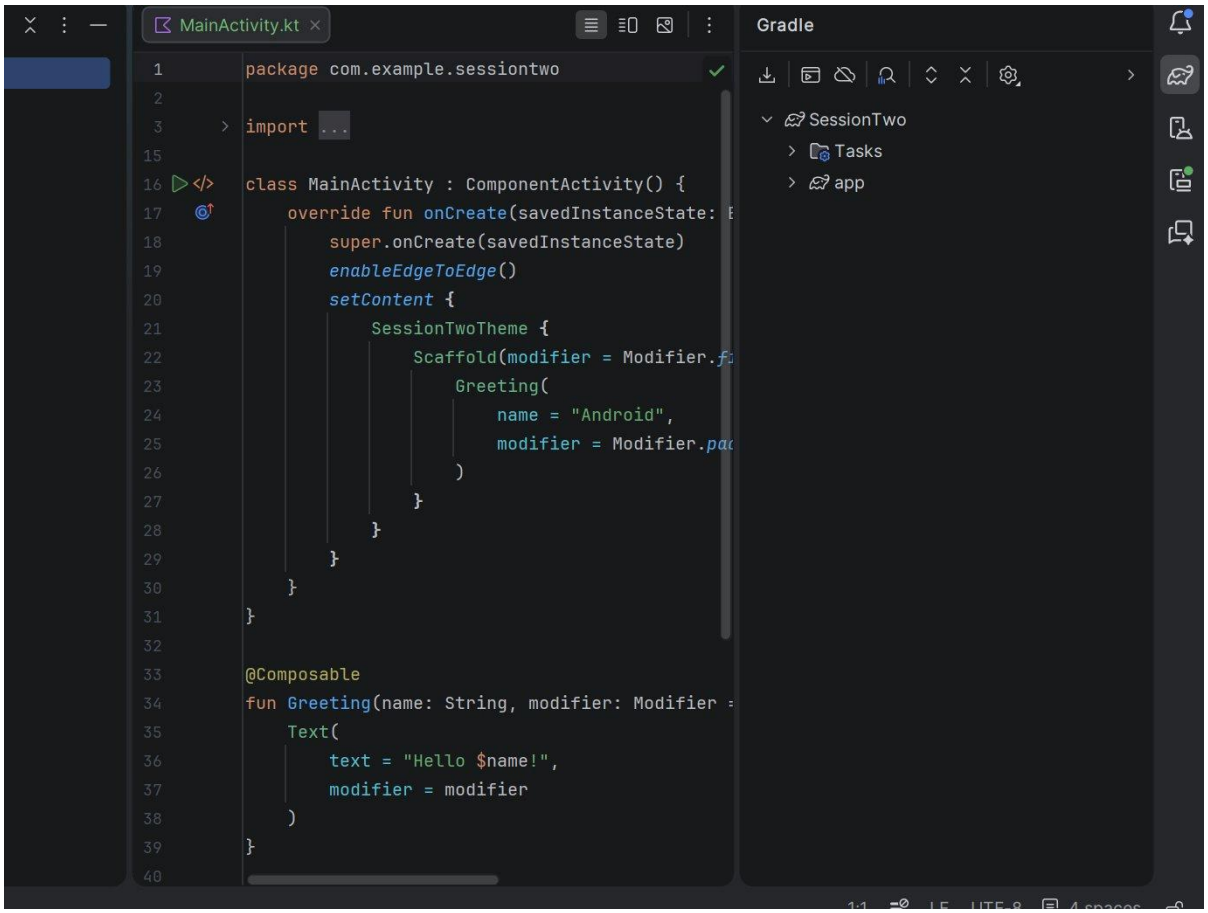

Figure 3 — MainActivity.kt showing setContent block and Greeting composable

Step 6 — Reviewing the @Preview Function

The GreetingPreview() composable was reviewed at the bottom of MainActivity.kt. This function is annotated with both @Preview and @Composable, allowing the Design panel to render the UI without launching the emulator.

Figure 4 — @Preview annotation and GreetingPreview composable function

Step 7 — Emulator Launch and First Run

The Pixel 6 AVD was launched from the Device Manager. The Run button was clicked and the app was deployed successfully. The screenshot below shows the Business Card application running on the emulator, displaying the name Guru, the title Android Developer Extraordinaire, and contact details.

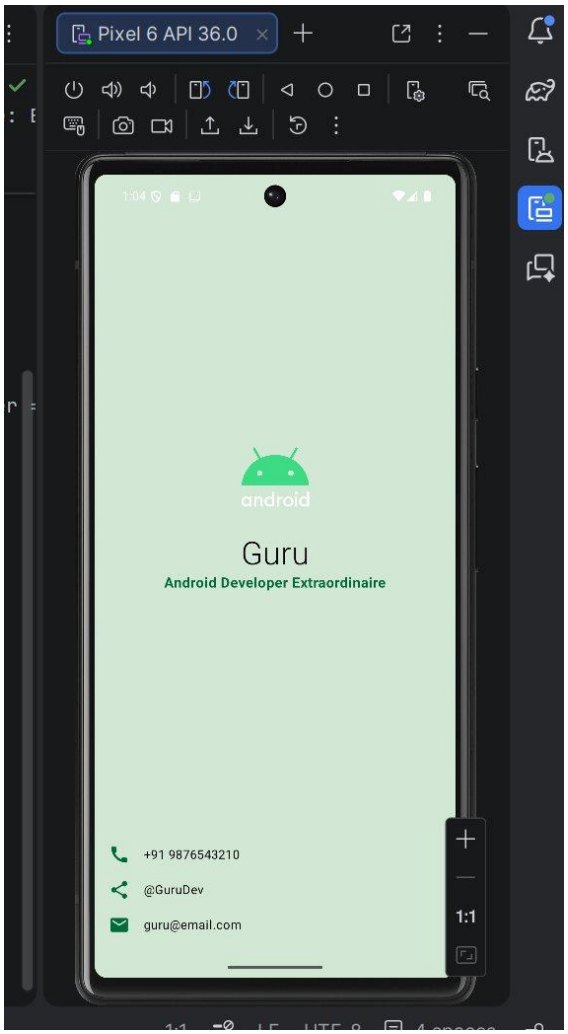

Figure 5 — App running on Pixel 6 AVD (API 36) showing Business Card UI

3. Code Concepts Explained

3.1 The setContent {} Block

The setContent {} block is the entry point for the Compose UI in an Activity. It replaces the traditional setContentView() method used in XML-based layouts. Within this block, composable functions are called to define and build the entire UI tree. The Compose runtime processes this lambda to render the interface on screen during the Activity's onCreate() lifecycle method.

3.2 The @Composable Annotation

The @Composable annotation marks a function as a composable unit in Jetpack Compose. Functions annotated with @Composable can emit UI elements, call other composable functions, and participate in the Compose recomposition lifecycle. The Compose compiler plugin processes this annotation at build time, enabling recomposition, state tracking, and efficient UI updates.

3.3 The @Preview Annotation

The @Preview annotation enables developers to render a composable function directly within Android Studio without deploying to a device or emulator. As shown in Figure 4, the GreetingPreview() function demonstrates this by wrapping the Greeting composable inside the SessionTwoTheme, providing an instant visual preview in the Design panel.

4. First Run Behaviour

When the application launches for the first time, the Android Runtime (ART) creates a new application process and invokes the MainActivity. The setContent {} block initiates the Compose composition process, traversing the tree of composable functions and producing drawing instructions. As visible in Figure 5, the resulting UI — the Business Card screen — is rendered cleanly on the Pixel 6 emulator, confirming that the Compose pipeline from source to screen is fully operational.

5. Lessons Learned

- **Gradle Sync Duration:** The initial Gradle sync took approximately four to five minutes due to downloading Compose-specific dependencies. Subsequent syncs are faster as Gradle caches artefacts locally.

- **Emulator Performance:** The Android Virtual Device exhibited occasional sluggishness during boot. Enabling Hardware Acceleration (HAXM or Hyper-V) and allocating additional RAM significantly improves emulator responsiveness.

- **Jetpack Compose vs. XML Layouts:** Compose's declarative, function-based approach eliminates separate XML files and findViewById() calls. The mental shift from imperative to declarative UI thinking requires deliberate practice but results in more concise, readable code.

6. Conclusion

The successful completion of this project setup task — evidenced by the screenshots captured throughout this report — demonstrates that the development environment is fully operational for Android development using Jetpack Compose. By creating a new project, performing a Gradle sync, reviewing the key source files, and executing a first run on the Android Virtual Device, the foundational workflow of the Android development lifecycle has been validated end-to-end.

The exercise reinforced an understanding of the core building blocks of a Compose-based application, including the setContent {} block, the @Composable annotation, and the @Preview utility. These concepts will form the foundation of all subsequent development tasks within this internship programme.

Overall, this task has established a strong baseline for proceeding with more complex Compose UI development, state management, and navigation topics in future sessions.